ANÁLISE DE ALGORITMOS DE ORDENAÇÃO

Central de distribuição de pedidos

ATIVIDADE PRÁTICA • ENGENHARIA DE SOFTWARE

Pietro Vianna da Rocha

1. Proposta do experimento

Na central de distribuição, organizar os pedidos por prioridade é uma tarefa simples de entender, mas que pode exigir muito processamento quando o volume aumenta. Neste trabalho, comparei Bubble Sort, Insertion Sort, Selection Sort e Quick Sort para observar como cada um reage ao receber a mesma sequência de códigos.

O foco não foi medir segundos, e sim contar comparações e alterações no vetor. Assim, a análise mostra o trabalho realizado pelos algoritmos sem depender diretamente da velocidade do computador.

2. Método e critérios de contagem

Dados: um único vetor aleatório para cada tamanho (10, 20 e 1.000), gerado com random.Random(42). Usei sample no intervalo de 1 a 100.000, sem repetição. A semente é iniciada uma vez, antes do laço dos tamanhos. Para cada execução, os quatro algoritmos recebem cópias idênticas e independentes.

Comparações (C): apenas testes entre valores do vetor ou entre um valor e a chave/pivô. A comparação que encerra a busca do Insertion também conta. Testes de índices, laços, pilha, validação e geração dos dados ficam fora da contagem.

Bubble e Selection: cada troca entre duas posições diferentes conta como 1 troca (T). No Quick, cada troca conta como 2 movimentações, correspondentes às duas escritas no vetor. Trocas de uma posição com ela mesma são evitadas.

Insertion: cada deslocamento conta como 1 movimentação; a escrita final da chave também conta, mesmo quando ela permanece na posição original. A cópia da chave para uma variável auxiliar não conta.

> **COMPARAÇÃO JUSTA | Para comparar o esforço total, uso escritas no vetor (E): E = 2T no Bubble e no Selection; E = movimentações no Insertion e no Quick. O indicador S = C + E é uma soma didática, não uma medição de tempo.**

Versões: Bubble com parada antecipada; Selection sem trocas desnecessárias; Insertion por deslocamentos; Quick iterativo, com partição de Lomuto e último elemento como pivô. A pilha evita o limite de recursão, mas não elimina o pior caso quadrático.

3. Resultados com vetores aleatórios

Os valores abaixo foram obtidos pela execução do programa incluído no apêndice. Todas as saídas foram comparadas com sorted(original), utilizado somente para validar o resultado, fora das contagens.

Tam.	Bubble C	Bubble T	Insertion C	Insertion M	Selection C	Selection T	Quick C	Quick M
10	45	26	33	35	45	6	29	20
20	175	87	101	106	190	15	65	54
1.000	498.015	243.076	244.070	244.075	499.500	994	10.234	9.756

C = comparações; T = trocas; M = movimentações. Bubble e Selection exibem T; Insertion e Quick exibem M, conforme o enunciado.

a) Menor número de comparações para 10 elementos

O Quick Sort realizou menos comparações: 29. O Insertion fez 33, enquanto Bubble e Selection fizeram 45 cada. Nesse vetor, o Quick precisou de menos verificações para chegar à mesma ordem final.

b) Menos trocas ou movimentações

O Selection Sort teve a menor contagem bruta nos três tamanhos: 6, 15 e 994 trocas. Mesmo convertendo tudo para escritas no vetor, ele continua com o menor valor: 12, 30 e 1.988 escritas. Para 1.000 elementos, os demais fizeram 486.152 (Bubble), 244.075 (Insertion) e 9.756 (Quick). Poucas trocas, porém, não significam poucas comparações.

c) O comportamento de 10 elementos se manteve com 20?

Em parte. O Quick continuou com menos comparações e o Selection continuou com menos escritas. A ordem por comparações foi Quick (65), Insertion (101), Bubble (175) e Selection (190). O empate entre Bubble e Selection, observado com 10 elementos, deixou de existir.

> **PRINCIPAL RESULTADO | Com 1.000 códigos aleatórios, o Quick fez 10.234 comparações contra 499.500 do Selection: aproximadamente 97,95% menos. A vantagem observada depende da entrada e da escolha do pivô.**

4. Interpretação dos resultados

d) O que aconteceu ao passar para 1.000 elementos?

As contagens aumentaram muito, sobretudo nos algoritmos quadráticos. De 20 para 1.000 elementos, o Bubble passou de 175 para 498.015 comparações; o Insertion, de 101 para 244.070; e o Selection, de 190 para 499.500. O Quick passou de 65 para 10.234. O vetor ficou 50 vezes maior, mas o trabalho não cresceu apenas 50 vezes.

e) Ter $O(n^2)$ significa fazer o mesmo número de operações?

Não. A complexidade descreve o crescimento, não uma contagem exata. Para 1.000 elementos aleatórios, Bubble, Insertion e Selection fizeram, respectivamente, 498.015, 244.070 e 499.500 comparações. O Selection sempre percorre a parte restante do vetor; o Bubble pode encerrar antes; e o Insertion para de procurar quando encontra a posição da chave. As constantes e a organização inicial também influenciam.

f) Qual apresentou maior crescimento?

Adotando o indicador S = comparações + escritas, de 10 para 1.000 elementos o Bubble teve o maior crescimento: de 97 para 984.167 operações (aumento de 984.070; cerca de 10.146 vezes). O Insertion passou de 68 para 488.145; o Selection, de 57 para 501.488; e o Quick, de 49 para 19.990.

Se o critério for somente comparações, o maior crescimento foi do Selection: de 45 para 499.500, aumento de 499.455. Por isso, é importante informar qual medida está sendo usada antes de apontar um vencedor.

g) Como o Quick Sort se diferenciou?

Nos vetores aleatórios, o Quick fez menos comparações nos três tamanhos e apresentou o menor S. Ele divide o problema em partições, em vez de percorrer repetidamente quase todo o vetor. Com 1.000 elementos, seu S foi 19.990, contra 984.167 do Bubble. Isso indica menor trabalho contado, mas não permite afirmar que ele foi 49 vezes mais rápido, pois não medi tempo.

h) Os resultados são coerentes com a teoria?

Sim. As entradas aleatórias mostraram o crescimento quadrático típico de Bubble, Insertion e Selection e um comportamento compatível com $O(n \log n)$ do Quick em média. Entretanto, três amostras não demonstram uma complexidade matemática. O desafio adicional também revelou os melhores casos lineares do Bubble otimizado e do Insertion e o pior caso $O(n^2)$ do Quick com pivô final.

i) Qual escolheria para milhares de pedidos?

Entre os quatro, escolheria o Quick Sort para lotes geralmente desordenados: com 1.000 códigos, ele fez 10.234 comparações e 9.756 escritas, totalizando 19.990 operações. Porém, não adotaria sem cuidados esta versão com pivô final se os pedidos pudessem chegar ordenados. Usaria uma estratégia de pivô mais robusta, como escolha aleatória, e novos testes. Essa melhoria é uma proposta, não um resultado medido neste trabalho.

5. Desafio adicional: a ordem inicial importa?

Repeti os testes nos três tamanhos. Para cada tamanho, usei os mesmos valores do vetor aleatório nas versões crescente e decrescente. Assim, muda apenas a organização inicial; cada algoritmo recebe sua própria cópia.

Tamanho / ordem	Bubble C / E	Insertion C / E	Selection C / E	Quick C / E
10 · aleatório	45 / 52	33 / 35	45 / 12	29 / 20
10 · crescente	9 / 0	9 / 9	45 / 0	45 / 0
10 · inverso	45 / 90	45 / 54	45 / 10	45 / 10
20 · aleatório	175 / 174	101 / 106	190 / 30	65 / 54
20 · crescente	19 / 0	19 / 19	190 / 0	190 / 0
20 · inverso	190 / 380	190 / 209	190 / 20	190 / 20
1.000 · aleatório	498.015 / 486.152	244.070 / 244.075	499.500 / 1.988	10.234 / 9.756
1.000 · crescente	999 / 0	999 / 999	499.500 / 0	499.500 / 0
1.000 · inverso	499.500 / 999.000	499.500 / 500.499	499.500 / 1.000	499.500 / 1.000

Em cada célula: C / E (comparações / escritas no vetor). Esta tabela usa a unidade normalizada; no Bubble e no Selection, as escritas equivalem ao dobro das trocas.

Análise do desafio

Não, a organização inicial não interfere da mesma maneira em todos. No vetor crescente de 1.000 elementos, o Bubble otimizado encerrou após 999 comparações e nenhuma troca. O Insertion também fez 999 comparações, mas registrou 999 escritas finais de chave, conforme o critério adotado. Ambos aproveitaram a ordem existente.

O Selection manteve 499.500 comparações nos três cenários de 1.000 elementos. Sua busca pelo menor percorre toda a região ainda não ordenada, mesmo que o vetor já esteja crescente. As escritas, por outro lado, mudaram: 1.988 no aleatório, zero no crescente e 1.000 no inverso.

O Quick passou de 10.234 comparações no aleatório para 499.500 tanto no crescente quanto no inverso. Como o pivô é sempre o último elemento, nesses casos ele fica em uma extremidade da ordem, produzindo partições desequilibradas. A versão iterativa não corrige esse problema.

No inverso, Bubble e Insertion chegaram a 499.500 comparações. O Bubble fez 999.000 escritas, enquanto o Insertion fez 500.499. Já o Quick fez só 1.000 escritas, apesar das 499.500 comparações: movimentar pouco não garante eficiência.

> **CONCLUSÃO | Não existe um vencedor independente dos dados e da implementação. Neste experimento, o Quick se destacou nos lotes aleatórios; Bubble e Insertion aproveitaram os lotes já ordenados; e o Selection reduziu movimentações sem reduzir sua busca quadrática.**

Apêndice A · Código Python

Execução: copie os blocos A, B e C, nesta ordem, para uma única célula do Google Colab ou para um arquivo Python e execute. Não há dependências externas. O programa imprime: tamanho | cenário | algoritmo | C | contador original | E | S.

Bloco A · Bubble, Insertion e Selection

```
import random


def bubble_sort(v):
    comparacoes = trocas = 0
    for fim in range(len(v) - 1, 0, -1):
        mudou = False
        for j in range(fim):
            comparacoes += 1
            if v[j] > v[j + 1]:
                v[j], v[j + 1] = v[j + 1], v[j]
                trocas += 1
                mudou = True
        if not mudou:
            break
    return comparacoes, trocas


def insertion_sort(v):
    comparacoes = movimentos = 0
    for i in range(1, len(v)):
        chave = v[i]
        j = i - 1
        while j >= 0:
            comparacoes += 1
            if v[j] <= chave:
                break
            v[j + 1] = v[j]
            movimentos += 1
            j -= 1
        v[j + 1] = chave
        movimentos += 1
    return comparacoes, movimentos


def selection_sort(v):
    comparacoes = trocas = 0
    for i in range(len(v) - 1):
        menor = i
        for j in range(i + 1, len(v)):
            comparacoes += 1
            if v[j] < v[menor]:
                menor = j
        if menor != i:
            v[i], v[menor] = v[menor], v[i]
            trocas += 1
    return comparacoes, trocas
```

Apêndice B · Quick Sort e execução justa

```
def quick_sort(v):
    # Lomuto, pivo final; pilha evita limite de recursao.
    comparacoes = movimentos = 0
    pilha = [(0, len(v) - 1)]
    while pilha:
        inicio, fim = pilha.pop()
        if inicio >= fim:
            continue
        pivo = v[fim]
        i = inicio
        for j in range(inicio, fim):
            comparacoes += 1
            if v[j] <= pivo:
                if i != j:
                    v[i], v[j] = v[j], v[i]
                    movimentos += 2
                i += 1
        if i != fim:
            v[i], v[fim] = v[fim], v[i]
            movimentos += 2
        pilha.append((inicio, i - 1))
        pilha.append((i + 1, fim))
    return comparacoes, movimentos


def executar(original):
    vetor_bubble = original.copy()
    vetor_insertion = original.copy()
    vetor_selection = original.copy()
    vetor_quick = original.copy()
    esperado = sorted(original)  # Apenas validacao externa.
    testes = [
        ("Bubble", bubble_sort, vetor_bubble),
        ("Insertion", insertion_sort, vetor_insertion),
        ("Selection", selection_sort, vetor_selection),
        ("Quick", quick_sort, vetor_quick),
    ]
    resultados = {}
    for nome, algoritmo, vetor in testes:
        comparacoes, contador = algoritmo(vetor)
        assert vetor == esperado, f"Falha: {nome}"
        escritas = 2 * contador if nome in (
            "Bubble", "Selection") else contador
        resultados[nome] = (comparacoes, contador, escritas)
    return resultados
```

Apêndice C · Geração e execução dos cenários

```
def experimento():
    rng = random.Random(42)
    dados = {}
    for n in (10, 20, 1000):
        original = rng.sample(range(1, 100001), n)
        cenarios = {
            "Aleatorio": original,
            "Ordenado": sorted(original),
            "Inverso": sorted(original, reverse=True),
        }
        dados[n] = {}
        if n <= 20:
            print("Vetor original", n, original)
        for cenario, vetor in cenarios.items():
            resultados = executar(vetor)
            dados[n][cenario] = resultados
            for nome, (comp, contador, escritas) in resultados.items():
                print(n, cenario, nome, comp, contador, escritas,
                      comp + escritas, sep=" | ")
    return dados


if __name__ == "__main__":
    experimento()
```

Reprodução e conferência

O gerador é iniciado com a semente 42 apenas uma vez. A ordem dos tamanhos é 10, 20 e 1.000. Reiniciar a semente dentro do laço ou trocar sample por randint produzirá outros vetores e poderá alterar as contagens. Para reproduzir este trabalho, mantenha o código como apresentado.

Vetor original de 10 elementos: [83811, 14593, 3279, 97197, 36049, 32099, 29257, 18290, 96531, 13435].

Vetor original de 20 elementos: [88697, 97081, 71483, 11396, 77398, 55303, 4166, 3906, 12281, 28658, 30496, 66238, 78908, 3479, 73564, 26063, 93851, 85182, 91925, 71427].

Validação: as 36 execuções do experimento (3 tamanhos × 3 cenários × 4 algoritmos) produziram a ordenação esperada. Uma conferência adicional das funções cobriu 880 entradas, incluindo permutações pequenas, vetor vazio, valores repetidos e sequências de 1.000 elementos já ordenadas ou invertidas.

Limite da análise: há uma amostra aleatória por tamanho, como solicitado. Os resultados descrevem essas entradas e estas versões dos algoritmos; não representam médias de muitas execuções nem um benchmark de tempo ou memória.